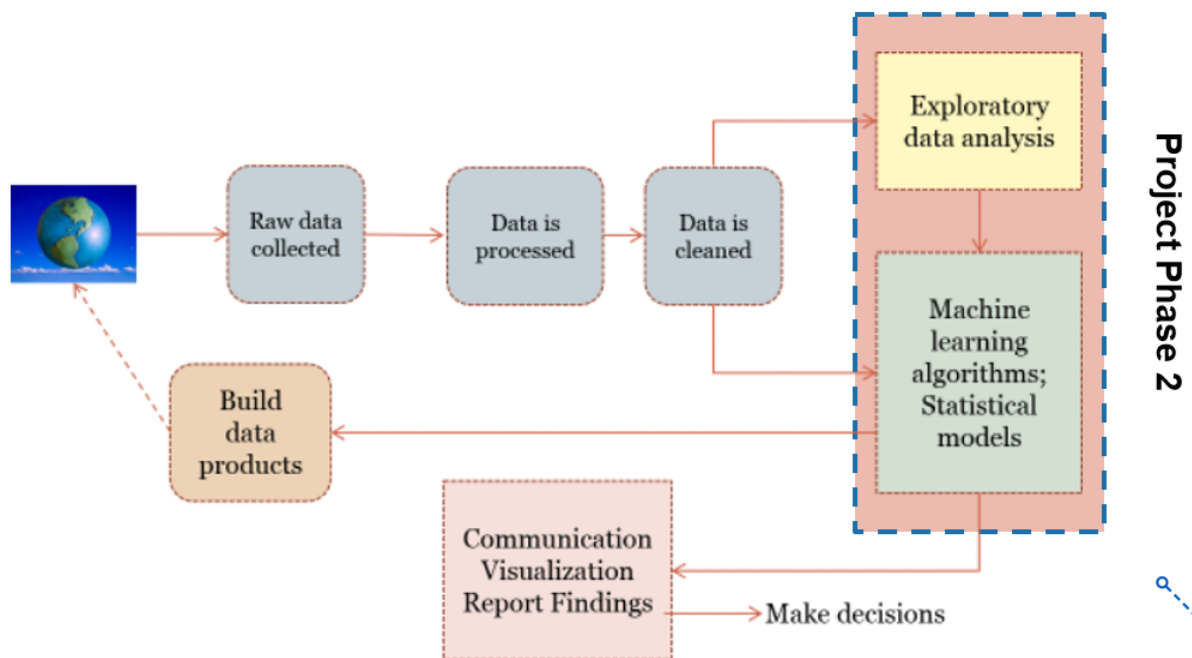


Project 1 Phase 2 & 3 – Data Modeling-Visualization

Due: 30th April



Learning Outcomes for Phase 2:

1. Identify suitable ML, MR, and/or statistical modeling algorithms. Model and apply algorithms to get insights into the behavior of the data. It could be classification, regression, clustering, etc.
2. Understand and explain the differences in each of the algorithms used.
3. Visualize the analytics using appropriate charts and graphs. You can use Seaborn or any other plotting tool.

Deliverables:

1. **Algorithms/Visualizations [25 points]:**
 - Apply 3 different significant and relevant algorithms (ML, MR, and/or statistical models) to your data and create visualizations for the results.
 - At least 1 of the 3 algorithms must be one that was not discussed in class.

- These outside algorithms can come from the class textbooks, or other sources. Cite the appropriate sources for each outside algorithm you choose to apply.
 - <https://scikit-learn.org/stable/>
 - <https://plotly.com/python/>
- 2. Explanation and Analysis [15 points]:**
- For each of the 3 above algorithms, provide justification for why you chose the particular algorithm, and discuss the effectiveness of the algorithm when applied to your data to answer questions related to your problem statement.
 - This should include discussion of any relevant metrics for demonstrating model effectiveness, as well as any intelligence you were able to gain from application of the algorithm to your data

References:

1. C. O'Neill and R. Schutt. Doing Data Science., O'Reilly. 2013.
2. NIST on EDA, <https://www.itl.nist.gov/div898/handbook/eda/section1/eda11.htm>
3. John Tukey Biography, <https://mathshistory.st-andrews.ac.uk/Biographies/Tukey/>
4. J. Tukey, http://theta.edu.pl/wp-content/uploads/2012/10/exploratorydataanalysis_tukey.pdf

Phase 3: Web Application for Model Inference

This phase would involve creating a simple web app where users can input new data, and your trained model can provide predictions based on that data. It can be hosted locally or online.

1. Choose Your Framework for the Web Application

There are a few different options for creating a web application. Here are some examples:

Flask (Python-based): A lightweight Python framework that is easy to use and ideal for small applications. It integrates well with Python-based machine learning models.

Streamlit: A very easy-to-use Python library for creating data apps with minimal effort. It allows you to deploy machine learning models and create user interfaces quickly.

Dash: A Python framework developed by Plotly for building analytical web applications. It's great for interactive data visualizations.

FastAPI: If you need higher performance and more flexibility, FastAPI is a Python framework that allows you to build APIs that can serve your model predictions.

2. Preparing Your Trained Model for the Web Application

Before building the web app, ensure your model is ready to be used for inference. You will need to:

Save your trained model: Depending on the library you used (e.g., `sklearn`, `TensorFlow`, `PyTorch`), you can save your trained model using a serialization format like `joblib` (for scikit-learn) or `pickle`. For deep learning models, you might use `model.save()` in Keras or save the model weights in TensorFlow.

```
import joblib

joblib.dump(model, 'my_model.pkl')
```

3. Building the Web Interface

You'll need to create a simple web form where users can input new data. Here's a basic outline of how you can build it:

Using Flask (Basic Example)

- Install Flask:

```
bash

pip install flask
```

- Create your Flask app:

```
from flask import Flask, render_template, request

import joblib

import numpy as np

app = Flask(__name__)
```

Load the trained model

```
model = joblib.load('my_model.pkl')

@app.route('/', methods=['GET', 'POST'])

def index():
```

```
    if request.method == 'POST':
```

Get form data from user

```
    data = [float(request.form['feature1']), float(request.form['feature2'])]
```

Convert data into a numpy array

```
input_data = np.array(data).reshape(1, -1)
```

Get prediction from the model

```
prediction = model.predict(input_data)
```

Return result to the user

```
return render_template('index.html', prediction=prediction)
```

```
return render_template('index.html', prediction=None)
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

Create a simple HTML form (index.html):

html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Model Prediction</title>
```

```
</head>
```

```
<body>
```

```
    <h1>Enter Data to Test Model</h1>
```

```
    <form method="POST">
```

```
        Feature 1: <input type="text" name="feature1"><br>
```

```
        Feature 2: <input type="text" name="feature2"><br>
```

```
        <input type="submit" value="Submit">
```

```
    </form>
```

```
{% if prediction %}
```

```
    <h2>Prediction: {{ prediction }}</h2>
```

```
{% endif %}  
</body>  
</html>
```

Running the app:

Once you have your Python file and HTML template ready, run the Flask app using:

```
bash
```

```
python app.py
```

This will start a local server (usually on `http://127.0.0.1:5000/`), where you can test your web interface. Users can input values and get predictions based on your model.

4. Testing with New Data

Once your web application is set up, users can input new data into the form and get predictions from the model. This is a great way to showcase your model's capabilities, as it allows others to interact with your work and see how the model performs on new, unseen data.

5. Optional: Hosting the Web App Online

If you want to make your web app accessible remotely, you can host it online using platforms like:

- Heroku: Free tier for small applications, easy integration with GitHub.
- PythonAnywhere: Another simple platform for hosting Python apps.
- Streamlit Cloud: If using Streamlit, this is a straightforward way to deploy the app.

Final Summary

1. Model Training and Evaluation: Completed in Phase 2.
2. Web Application: Developed in Phase 3 to test new data and showcase your model interactively.
 - Use Flask, Streamlit, or Dash for creating a simple web interface.
 - Allow users to input new data and get predictions.
3. Deployment (optional): Host the web app on platforms like Heroku, Streamlit Cloud, or PythonAnywhere to make it publicly accessible.

This approach allows you to demonstrate the practical application of your machine learning model in a way that's interactive and engaging, even if you're not deploying it in a full-scale production environment.